ChronosAI Companion

An Agentic Vision Companion for Elders Living Alone
OpenCV 5 and AWS Competition Entry — Agentic Vision Path

Koyyuru Yamini (solo project)

October 2026

Abstract

Many elderly people live alone, and a fall or a medical episode can stay unnoticed for hours. ChronosAI Companion is a camera-and-voice companion that *sees* (OpenCV 5), *asks* (a voice check-in ladder) and *acts* (an AWS-backed alert that a physician approves or rejects). Visual evidence decides what the system asks, how long it waits, and whether it escalates. A physician makes the final dispatch decision, so the system assists emergency care rather than replacing it. The prototype runs the OpenCV pipeline on a local computer and uses Amazon S3, Amazon SES, Amazon CloudWatch Logs and S3 lifecycle rules for the cloud side.

1 Problem and users

A person living alone who falls, faints or becomes unwell may not be able to call for help. Wearable alarms are often forgotten or not worn, and cameras that only record do not act. Loneliness is a second, separate problem: many elders spend long hours without anyone to talk to. ChronosAI Companion addresses both. It keeps the person company (greetings, quizzes, routine reminders for meals and medicines) and watches over their safety.

Primary users: elderly people living alone. **Secondary users:** family members and physicians who receive an actionable alert (risk level, summary, snapshot) and can dispatch help or mark a false alarm.

According to the World Health Organization, an estimated 684 000 people die from falls each year, making falls the second leading cause of unintentional injury death after road traffic injuries. Adults older than 60 suffer the greatest number of fatal falls, and about 37.3 million falls each year are severe enough to need medical attention.[1]

2 System overview

Figure 1 shows the components. Perception and the decision logic run locally with OpenCV 5 (version 5.0.0). When the person cannot confirm they are safe, the agent uploads one snapshot and an incident record to Amazon S3, generates a review page for the physician, and sends an email through Amazon SES. The physician's decision is stored in S3 and the robot reads it back.

3 OpenCV 5 implementation

The same module (`perception.py`) is used by the live application and by the offline evaluation, so the reported numbers describe the demo behaviour.

[1] WHO, *Falls* fact sheet, https://www.who.int/news-room/fact-sheets/detail/falls, accessed October 2026.

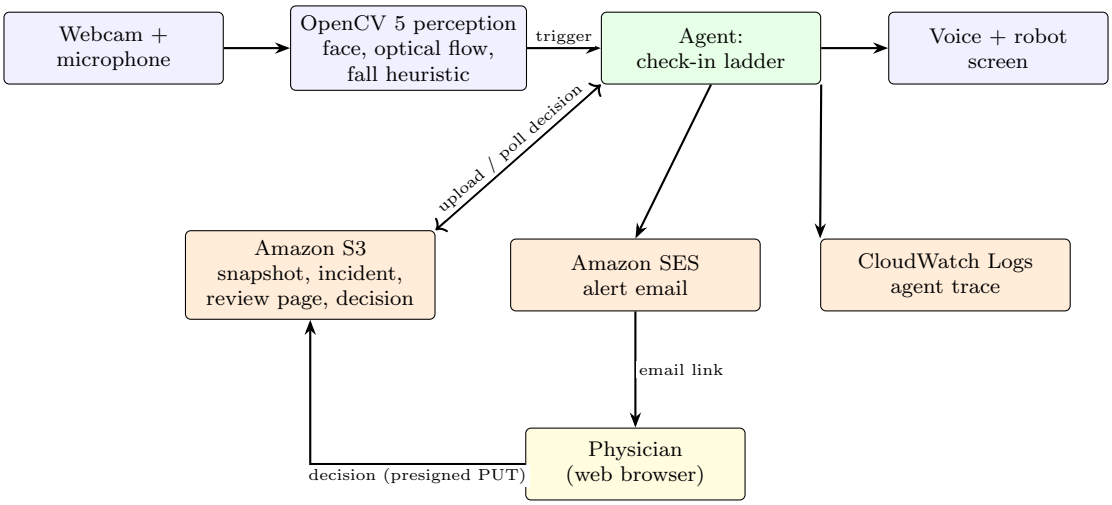


Figure 1: Architecture. Orange boxes are AWS services; blue boxes run on the local computer.

- **Face detection:** YuNet through `cv2.FaceDetectorYN` (OpenCV DNN), with a Haar cascade as fallback.
- **Motion:** dense Farneback optical flow (`cv2.calcOpticalFlowFarneback`) on 160 × 120 frames. The motion value is the fraction of pixels moving faster than 0.5 px per frame.
- **Immobility:** a face is present and motion stays below 0.01 for N seconds.
- **Fall heuristic:** the face centre drops by more than 1.2 face-heights inside a 1.5 s window together with strong motion, and afterwards the person is absent or still for at least 1.5 s (observed 1–5 s after the drop).
- **Face lost:** the person was seen for at least 5 s, then disappears for N seconds and was last seen in the lower part of the frame.

Trigger	Live demo setting	Short-clip setting (`evaluate.py`)
Immobility threshold N	30 s (8 s in demo mode)	8 s
Face-lost threshold N	15 s (6 s in demo mode)	5 s

Table 1: Timer settings. Short timers are used only for clips and for the video.

4 Agentic behaviour

Each trigger from the vision module changes what the agent does next (Table 2). A chatbot that only describes a fixed vision result would not qualify; here the vision evidence selects the question, the waiting time and whether movement alone counts as an answer.

Vision trigger	First question	Wait	Movement counts as answer
FALL_SUSPECTED	"Are you hurt?"	6 s	no
IMMOBILE	"Have you not moved for a while?"	8 s	yes (at least 0.5 s of motion)
FACE_LOST	"I cannot see you anymore."	10 s	no
VOICE_DISTRESS	(skips questions)	—	—

Table 2: Different visual evidence produces a different first action.

Human control. The robot never dispatches help by itself. The physician opens the review page and chooses "Dispatch" or "False alarm". If no decision arrives, a reminder email is sent after half

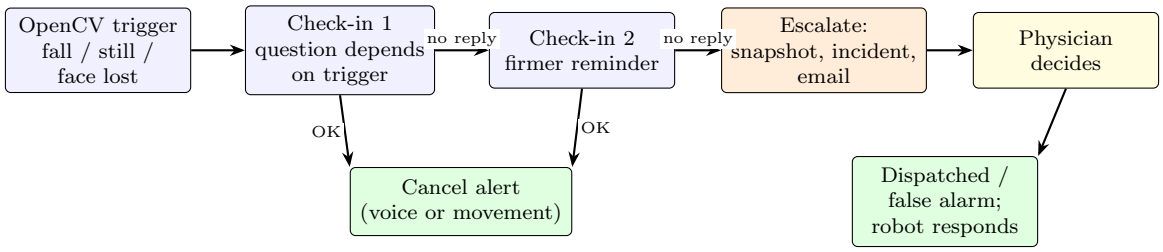


Figure 2: Agent workflow: perception, decision, action. The patient can also cancel by voice while waiting for the physician.

of the waiting time (waiting time 180 s, 90 s in demo mode) and the robot asks the person to call an emergency contact.

Voice distress. Words such as "help" or "not well" go straight to escalation, even during a privacy pause.

Observability. Every step is written to `agent_trace.jsonl` and sent to Amazon CloudWatch Logs. Excerpt of a real run (immobility simulated with a hotkey, so only voice counted as an answer):

```
20:26:36 trigger          IMMOBILE
20:26:53 checkin_result   NO_RESPONSE   (heard: "")
20:27:11 checkin_result   NO_RESPONSE   (heard: "")
20:27:15 decision         escalate_to_physician, risk HIGH
20:28:02 decision         reminder_email_sent
```

In another run the physician pressed "Dispatch"; the trace then shows `physician_decision DISPATCHED` and the robot said that help is on the way.

5 AWS deployment

Service	Use in ChronosAI Companion
Amazon S3	One snapshot (JPEG, at most 640 px wide, server-side encrypted) per escalation; incident record; the physician review page; the physician's decision (a small object written through a presigned PUT URL); the robot polls this prefix.
S3 lifecycle rule	Rule `expire-old-data` deletes objects after 7 days (rules run once per day, so deletion happens about one week later).
Amazon SES	Alert and reminder emails with a link to the review page.
Amazon CloudWatch Logs	Structured agent trace for monitoring and for judges (`/chronos/agent`).

Table 3: AWS services actually used.

Security. The bucket blocks all public access. Review links are presigned and expire after three days. All displayed values are HTML-escaped. Credentials are loaded from a `.env` file that is not committed to the repository.

What we could not use. On our account, DynamoDB, Lambda and SNS returned "explicit deny in a service control policy" during most of the build period, and Amazon Bedrock model calls returned "Operation not allowed". Because of this, incident storage uses S3 instead of DynamoDB, the physician page is a static page served through presigned URLs instead of a Lambda function, and the risk level is produced by simple rules instead of a language model. The code keeps a Gmail SMTP fallback if SES is unavailable.

6 Companion features and privacy by design

The robot can chat, run a short quiz, mention songs and news, and remind the person about meals and medicines. Because constant monitoring can feel intrusive, the person can say "I need a break" (pause) or "good night" (sleep). During a break all triggers are ignored. During sleep, stillness is ignored but falls are still watched. Both modes end automatically (30 minutes and 8 hours) so monitoring can never stay off by accident, and "help" always cancels the pause and raises an alert.

7 Evaluation

Status. Because of limited time we did not complete a formal clip-based evaluation, so this report gives no accuracy or false-alarm percentages. We tested the whole system interactively on a laptop webcam in demo mode (short timers), using real stillness in front of the camera, hotkey-simulated triggers (fall, immobility) and spoken answers, and we checked the AWS side by inspecting the stored objects and logs. Table 4 lists what we observed.

Scenario	Observed outcome
Immobility trigger, no answer	Two questions with no reply, then escalation with risk HIGH (trace entries `checkin_result` twice, then `escalate_to_physician`).
Escalation	Snapshot, incident record and review page stored in Amazon S3; alert email delivered through Amazon SES (Gmail SMTP as fallback).
Physician presses "Dispatch"	Review page showed the snapshot and recorded the decision; the robot read it from S3 (`physician_decision DISPATCHED`) and said that help is on the way.
Physician presses "False alarm"	Decision recorded; the robot resumed normal monitoring.
Voice commands	"I need a break", "help" and "I am back" were recognised; "help" during a pause raised an alert and cancelled the pause.
Observability	Each decision appeared in `agent_trace.jsonl` and as a log stream in CloudWatch Logs.

Table 4: Observed behaviour in live tests.

Measuring accuracy. The repository includes `record_clip.py` and `evaluate.py`. They replay recorded clips through the same perception module as the live app and report trigger accuracy, false-alarm rate on normal clips, and processing time per frame. We did not run them on a full clip set, so we report no such numbers; this is the most important next step.

8 Limitations

- The fall detector is a heuristic, not a trained pose model. We did not complete a quantitative evaluation on recorded clips, so no accuracy or false-alarm rates are reported.
- Face detection fails when the face is turned away, covered or poorly lit; several people in view are not handled.
- Leaving the camera view for the face-lost time starts a check-in. This is right for a person who collapsed out of view, but a false alarm for someone who simply walked away (in a live test on 26 October this happened after 6 s with the demo timer). Requiring the last position to be low in the frame reduces this but does not remove it.
- Speech recognition uses Google's service (internet required) and works only in English. The microphone listens in 4-second windows with a 5-second gap, so a spoken word such as "help" can be missed; the camera check-in and the physician's approval are the other safeguards.

- The OpenCV processing runs on the local computer; the cloud side is storage, email and logging.
- Some AWS services were unavailable on our account (see above).
- This is a research prototype, not a medical device, and it must not replace emergency services.

Issues found and fixed during testing. An early version could count the movement of reaching for the keyboard as the patient answering; movement is now measured only after the robot finishes speaking and must last about half a second, and for fall and face-lost triggers movement alone is never accepted. A code review found that the first pause version ignored "help" and left monitoring disarmed afterwards; distress now always overrides a pause, pauses end automatically, and falls are still watched during sleep.

9 Responsible use

- **Consent:** the elder (and family) must opt in; the robot announces what it does.
- **Data minimisation:** no continuous recording; one snapshot only when an alert is escalated; objects and review links expire automatically.
- **Audio:** recognised speech is sent to Google's speech service; users must be told, and Amazon Transcribe is a future alternative. While paused, the microphone still listens for "I am back" and "help" for safety.
- **False alarms:** the robot asks first, and a physician decides before any dispatch.
- **Bias and access:** English-only voice and a single test subject limit who this works for today.

10 Future work

Trained pose estimation for falls; multilingual voice (Telugu, Hindi) with matching text-to-speech; moving the review page and incident log to Lambda and DynamoDB, and risk summaries to Amazon Bedrock, once those services are available on the account; running the OpenCV pipeline on AWS (for example, a container) for multi-camera homes; a larger and more varied evaluation. The same see–ask–escalate loop could also watch a driver: if the driver leaves the driver's-seat region or stops responding, the system warns the driver and, with consent, notifies an emergency contact or doctor. This needs separate in-vehicle testing (lighting, camera placement, latency) and careful safety design.

11 Reproducibility

The repository README lists the demo video, setup, run and test steps. Setup: `pip install -r requirements.txt`, copy `.env.example` to `.env`, run `python download_models.py`, then `python chronos_companion.py --demo`. Evaluation: `python record_clip.py <name> 18` for each clip, then `python evaluate.py --dir clips --immobile 8 --facelost 5`. Pinned dependencies are in `requirements.txt` (generated with `pip freeze`).